

Driving Motors

Experiment Objective

This section will guide you through driving DC geared motors with the MicroROS-V2 control board. Through this tutorial, you will learn how to use the ESP32-S3's MCPWM peripheral together with the `bdc_motor` component to control the forward/reverse direction and speed of the motors, implementing basic control such as acceleration, braking, and coasting stop.

Experiment Principle

1. Keywords

Keyword	Description
DC Geared Motor	A DC motor with a built-in reduction gearbox that provides high torque; speed is controlled by regulating the voltage through PWM
MCPWM	The ESP32-S3's motor control PWM peripheral, which can configure multiple groups of timers and generators to drive DC motors together with the <code>bdc_motor</code> component
<code>bdc_motor</code>	Espressif's official DC motor driver component, which encapsulates MCPWM configuration and forward/reverse, brake, and coast control logic
Brake stop (STOP_BRAKE)	The motor coil is short-circuited, producing electromagnetic damping that stops the motor immediately
Coast stop (STOP_COAST)	The motor coil is disconnected, and the motor decelerates to a stop by inertia
Dead Zone	The region where the motor cannot rotate when the PWM duty cycle is too low; the code automatically adds a bias value (200) to cross the dead zone

2. Technical Architecture

The program uses the ESP32-S3's MCPWM peripheral together with Espressif's official `bdc_motor` component to drive two DC geared motors (M1 and M2). M1 uses GPIO13 (PWM A) and GPIO14 (PWM B); M2 uses GPIO11 (PWM A) and GPIO12 (PWM B). The PWM frequency is 25KHz and the timebase resolution is 10MHz. The code implements logic direction mapping (M2 is inverted due to mirrored mounting), dead zone filtering, and limit protection, providing basic control functions such as forward, reverse, brake stop, and coast stop.

Main component files:

- `main/main.c` — main program entry, demonstrates the forward / brake / reverse / coast cycle
- `components/pwm_motor/pwm_motor.c` / `pwm_motor.h` — motor driver implementation and interface declarations

Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✓ Required
USB data cable	✓ Required (for power supply and serial monitor)
Battery (12V)	✓ Required (to power the motor driver)
MicroROS-V2 chassis motors	✓ Required (connected to motor interfaces M1 / M2)

2. Prerequisites

1. ESP32-IDF development environment set up
2. MicroROS-V2 control board connected to the PC via USB
3. Battery connected to the control board power interface

Motor wiring description

The motor driver interfaces on the MicroROS-V2 control board are defined as follows:

Motor interface	Pin A	Pin B
M1	GPIO14 (PWMA)	GPIO13 (PWMB)
M2	GPIO12 (PWMA)	GPIO11 (PWMB)

Note: The physical wiring directions of M1 and M2 are opposite, which is already handled in the code through logic direction constants.

3. Compile and Flash

```
cd ~/esp/base_samples/pwm_motor
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

After the program starts, the motors execute the following action cycle in sequence:

1. Motors rotate forward (forward direction) at maximum speed
2. Brake stop (STOP_BRAKE)
3. Motors rotate reverse (backward direction) at maximum speed
4. Coast stop (STOP_COAST)

Each state lasts 1 second and the cycle repeats.

4. How to Stop

Press `Ctrl+]` in the terminal to exit the serial monitor.

Experiment Result

After correctly flashing and running the program, the serial monitor outputs the following logs:

```
hello yahboom
I (xxx) MAIN: Nice to meet you!
I (xxx) PWM_MOTOR: Init PwmMotor Device
I (xxx) MAIN: Motor Forward
I (xxx) MAIN: Motor Stop Brake
I (xxx) MAIN: Motor backward
I (xxx) MAIN: Motor Stop Coast
I (xxx) MAIN: Motor Forward
...
```

The DC motors connected to the M1 and M2 interfaces execute forward, brake stop, reverse, and coast stop in sequence according to the log order, with each state lasting 1 second.

- **Forward/Reverse:** the motors rotate at maximum speed
- **Brake stop:** the motors stop rotating immediately
- **Coast stop:** the motors gradually decelerate to a stop by inertia

Code Explanation

Source code paths:

- Firmware source code: `~/esp/base_samples/pwm_motor/main/main.c`
- Motor driver: `~/esp/base_samples/pwm_motor/components/pwm_motor/pwm_motor.h`,
`pwm_motor.c`

1. Project Structure

```
pwm_motor/
├── CMakeLists.txt           # Top-level CMake project file
├── main/
│   ├── CMakeLists.txt      # main component build configuration
│   └── main.c              # Main program entry
├── components/
│   └── pwm_motor/
│       ├── CMakeLists.txt  # Component configuration (depends on driver,
bdc_motor)
│       ├── idf_component.yml # Component dependency declarations (bdc_motor:
^0.1.0)
│       ├── pwm_motor.c     # Motor driver implementation
│       └── pwm_motor.h     # Motor driver interface declarations
└── managed_components/
    └── espressif__bdc_motor/ # bdc_motor component (automatically pulled in)
        └── include/bdc_motor.h
```

└─ src/bdc_motor.c

The `bdc_motor` component is the official DC motor driver library provided by Espressif. During compilation, it is automatically pulled from the component registry according to `idf_component.yml`.

2. Main Program Logic

The execution flow of `app_main` in `main.c`:

1. Print the startup information
2. After a 1-second delay, call `PwmMotor_Init()` to initialize the motor driver
3. Enter the main loop, executing in sequence:
 - `PwmMotor_Set_Speed_All(speed, speed)` — both wheels rotate forward at maximum speed
 - `PwmMotor_Stop(MOTOR_ID_ALL, STOP_BRAKE)` — both wheels brake stop
 - `PwmMotor_Set_Speed_All(-speed, -speed)` — both wheels rotate reverse at maximum speed
 - `PwmMotor_Stop(MOTOR_ID_ALL, STOP_COAST)` — both wheels coast stop

3. Key Components

Motor Driver (`pwm_motor.h` / `pwm_motor.c`)

Pin and PWM parameter definitions:

Parameter definition file: `~/esp/base_samples/pwm_motor/components/pwm_motor/pwm_motor.h`

```
// Motor driver GPIO
#define PWM_GPIO_M1A      13      // M1 motor phase A
#define PWM_GPIO_M1B      14      // M1 motor phase B
#define PWM_GPIO_M2A      11      // M2 motor phase A
#define PWM_GPIO_M2B      12      // M2 motor phase B

// Logic direction (the forward direction of the car is the positive direction)
#define PWM_MOTOR_LOGIC_SIGN_M1  (+1)  // M1 positive direction
#define PWM_MOTOR_LOGIC_SIGN_M2  (-1)  // M2 reverse direction (mirrored mounting)

// PWM clock and frequency
#define PWM_MOTOR_TIMER_RESOLUTION_HZ  10000000  // 10MHz, 1 tick = 0.1us
#define PWM_MOTOR_FREQ_HZ              25000    // PWM frequency 25KHz
#define PWM_MOTOR_DUTY_TICK_MAX        (400)    // Theoretical maximum duty cycle ticks
(10MHz / 25KHz)
#define PWM_MOTOR_DEAD_ZONE            (200)    // Dead zone filtering value
#define PWM_MOTOR_MAX_VALUE            (200)    // Maximum effective speed value (400 -
200)
```

The logic direction of M2 is inverted (`-1`) because the two motors are mounted in a mirror configuration on the car chassis; reverse direction control is needed for them to drive in the same direction.

Enumeration definitions:

```
typedef enum _motor_id {
    MOTOR_ID_ALL = 0,    // All motors
    MOTOR_ID_M1 = 1,     // Motor M1
    MOTOR_ID_M2 = 2,     // Motor M2
} motor_id_t;

typedef enum _stop_mode {
    STOP_COAST = 0,      // Coast stop (motor coil disconnected, free rotation)
    STOP_BRAKE = 1       // Brake stop (motor coil short-circuited, immediate stop)
} stop_mode_t;
```

Motor initialization (`PwmMotor_Init_M1` / `PwmMotor_Init_M2`):

Initialize the motor using the `bdc_motor` component:

```
bdc_motor_config_t motor_config = {
    .pwm_freq_hz = PWM_MOTOR_FREQ_HZ,           // 25KHz
    .pwma_gpio_num = PWM_GPIO_M1B,             // M1's PWMA is connected to GPIO14
    .pwmb_gpio_num = PWM_GPIO_M1A,             // M1's PWMB is connected to GPIO13
};

bdc_motor_mcpwm_config_t mcpwm_config = {
    .group_id = PWM_MOTOR_TIMER_GROUP_ID_M1,    // Timer group ID (M1 and M2 are both group 0)
    .resolution_hz = PWM_MOTOR_TIMER_RESOLUTION_HZ, // 10MHz
};

bdc_motor_new_mcpwm_device(&motor_config, &mcpwm_config, &motor);
bdc_motor_enable(motor);
```

M2's pin configuration is PWMA=GPIO12, PWMB=GPIO11, consistent with the A/B phase order of M1.

Speed control function (`PwmMotor_Apply_To_Device`):

The low-level speed setting function, with parameters `bdc_motor_handle_t` and the speed value:

```
static void PwmMotor_Apply_To_Device(bdc_motor_handle_t motor, int speed)
```

Processing flow:

1. `PwmMotor_Limit_Speed` — limits to `[-PWM_MOTOR_DUTY_TICK_MAX, PWM_MOTOR_DUTY_TICK_MAX]`
2. `speed > 0`: call `bdc_motor_forward` for forward rotation + `bdc_motor_set_speed` to set the speed
3. `speed < 0`: call `bdc_motor_reverse` for reverse rotation + `bdc_motor_set_speed` to set the speed (absolute value)
4. `speed == 0`: choose `bdc_motor_brake` (brake) or `bdc_motor_coast` (coast) according to the `stop_brake` flag

Dead zone filtering (`PwmMotor_Ignore_Dead_Zone`):

There is a dead zone near a speed value of 0; the motor cannot rotate when the PWM duty cycle is too low. The code automatically adds `PWM_MOTOR_DEAD_ZONE` (200) to the speed value to ensure the speed command can effectively drive the motor:

```
static int PwmMotor_Ignore_Dead_Zone(int speed)
{
    if (speed > 0) return speed + PWM_MOTOR_DEAD_ZONE;
    if (speed < 0) return speed - PWM_MOTOR_DEAD_ZONE;
    return 0;
}
```

Logic direction mapping (`PwmMotor_Apply_Logic_Direction`):

Apply the logic direction sign according to the motor ID; M1 is positive (+1) and M2 is negative (-1):

```
static int PwmMotor_Apply_Logic_Direction(motor_id_t motor_id, int speed)
{
    int logic_sign = 1;
    if (motor_id == MOTOR_ID_M1)      logic_sign = PWM_MOTOR_LOGIC_SIGN_M1;
    else if (motor_id == MOTOR_ID_M2) logic_sign = PWM_MOTOR_LOGIC_SIGN_M2;
    return speed * logic_sign;
}
```

API functions:

Function	Description
<code>void PwmMotor_Init(void)</code>	Initialize the two-channel motor driver
<code>void PwmMotor_Set_Speed_All(int speed_1, int speed_2)</code>	Set the speeds of M1 and M2 simultaneously
<code>void PwmMotor_Set_Speed(motor_id_t motor_id, int speed)</code>	Set the speed of the specified motor
<code>void PwmMotor_Stop(motor_id_t motor_id, bool brake)</code>	Stop the specified motor; <code>true</code> for brake, <code>false</code> for coast

A positive speed value indicates forward rotation (forward), and a negative value indicates reverse rotation (backward). The speed range is `[-200, 200]` (after dead zone filtering, the actual PWM duty cycle is `0~400`).

Common Problems

Problem	Cause	Solution
Motors do not rotate	Battery not connected or poor motor wiring	Confirm the battery is connected to the control board power interface and check whether the motor wiring is plugged in firmly

Problem	Cause	Solution
Motors only rotate in one direction	Wrong logic direction configuration or M1/M2 wiring reversed	Check the value of <code>PWM_MOTOR_LOGIC_SIGN_M2</code> and whether the motor wires are inserted in reverse
Motors jitter or make sound without rotating	Dead zone filtering value too small or insufficient battery voltage	Check the <code>PWM_MOTOR_DEAD_ZONE</code> value, or use a freshly charged battery
Motor forward direction is opposite to expected	Logic direction constants need adjustment	Modify the signs of <code>PWM_MOTOR_LOGIC_SIGN_M1</code> and <code>PWM_MOTOR_LOGIC_SIGN_M2</code>